

Prompt-Engineered Extraction of Original Vector Trait Metadata from Scientific PDFs: A Provenance-Aware Workflow for VecTraits Curation

Abstract

1 Introduction

Vector-borne diseases (VBDs) remain a persistent and expanding challenge for human, animal and plant health, and their dynamics are strongly shaped by the biological traits of vectors. Traits such as development time, survival, longevity, fecundity, feeding behaviour and transmission-related characteristics affect not only pathogen transmission directly, but also vector abundance, life history and responses to environmental change [Cator et al., 2020]. Trait-based approaches are therefore increasingly central to understanding variation in disease transmission and to building ecological and epidemiological models of vector-borne disease systems.

Despite their importance, vector trait data remain difficult to synthesise and reuse at scale. One major obstacle is that relevant measurements are reported heterogeneously across the literature: trait names differ between studies, definitions are often incomplete or inconsistent, units may be omitted or expressed in different ways, and key information can be distributed across narrative text, tables and figures. These inconsistencies mean that vector trait data, as typically reported in the literature, fail to meet the FAIR principles of being findable, accessible, interoperable and reusable, including by machines [Wilkinson et al., 2016] directly limiting the ability to integrate published measurements into shared, reusable databases. Similar problems have long been recognised more broadly in ecology, where inconsistent terminology and weakly formalised concepts reduce the discoverability and interpretability of data, motivating clearer definitions, stronger semantic structure and ontology-informed data practices [Madin et al., 2008]. Structured, provenance-aware extraction from the published literature is therefore a prerequisite for building vector trait resources that meet these standards and can support downstream ecological and epidemiological modelling.

VecTraits is a repository focused on trait data for vectors, pathogens and hosts relevant to VBDs [Cator et al., 2026]. It provides an important infrastructure for compiling trait measurements from the literature, but its continued development depends on the ability to recover information from a large and heterogeneous body of published papers. Over the last several years, the project has curated 334 papers with over 40,000 rows of trait data. Because these records were compiled across many studies and curation stages, some fields are not always populated in a fully consistent way. For example, original

trait name (OTN) and original trait definition (OTD) may not always preserve the exact source-paper wording, while standardised fields such as units, life stage, experimental conditions and error terms may be incomplete or inconsistently recorded across studies.

This focus on original reporting is also consistent with recent efforts to improve vector trait data standards. MIREVTD, a Minimum Information standard for Reporting Vector Trait Data, was proposed to make vector trait data more consistent, reusable and easier to compare across studies, especially where terminology and metadata are inconsistent or incomplete [Ryan et al., 2025]. However, such standards mainly guide how future studies should report trait data. They do not directly solve the problem of extracting useful records from already-published papers, where traits may have been reported using different names, definitions, units and levels of detail.

For existing literature, a practical intermediate workflow is therefore still needed. First, trait information should be extracted as closely as possible to how it was originally reported in the paper. These candidate records can then be checked, reviewed and, where appropriate, linked to broader standardised trait names or definitions. In this sense, extracting original trait metadata is not separate from standardisation. Instead, it provides the necessary starting point for reliable standardisation.

Recent advances in large language models (LLMs) offer a possible route for accelerating this extraction task. Few-shot and instruction-based prompting have shown that large models can perform complex language tasks with little or no task-specific training data, making them attractive for information-extraction workflows where labelled examples are limited [Brown et al., 2020]. Multimodal models, which can process both text and images, are particularly relevant here because scientific PDFs often contain information embedded in figures and tables that is not recoverable from machine-extracted text alone.

However, scientific information extraction remains challenging because papers contain specialised terminology, heterogeneous structure and context-dependent reporting styles. Relevant information may be split across prose, tables and figures, and the same biological concept may be described in different ways between studies. In trait databases specifically, extraction errors propagate silently into downstream models: a mislabelled unit, a misassigned life stage or an incorrectly grouped measurement can affect quantitative analyses that depend on the accuracy of individual field values [Nasar et al., 2018]. These risks make systematic evaluation necessary before LLM-extracted records can be used as reliable inputs for trait databases or downstream modelling.

The overall objective of this project was to build and evaluate a reproducible, provenance-aware pipeline for extracting original vector trait metadata from scientific PDFs, with the aim of supporting VecTraits curation. I assessed whether an off-the-shelf multimodal large language model could be prompt-engineered to recover schema-constrained, record-level trait information from published literature accurately enough to support, but not replace, expert curation. Rather than treating model outputs as final database entries, this project evaluated them as candidate records requiring validation and review. By comparing extracted records with manually curated reference data, the project examined where AI-assisted extraction succeeded, where it failed, and whether iterative prompt refinement could make the workflow practically useful for vector trait-data curation. In doing so, it contributes both a practical extraction-and-review workflow and a methodological assessment of how AI-assisted extraction can support FAIR-oriented ecological data infrastructure [Wilkinson et al., 2016, Madin et al., 2008].

2 Methods

This project developed a reproducible, human-in-the-loop extraction and validation workflow for recovering vector trait metadata from scientific PDFs in the context of VecTraits. The workflow comprised six connected stages: assembly of a stratified paper corpus and manual gold benchmark, definition of a reduced extraction schema, input preparation and model inference, prompt-guided record extraction, structured output validation, and quantitative evaluation with qualitative discrepancy analysis (Fig. 1). Prompt engineering was treated as an iterative component of the workflow, with recurring errors from evaluation and discrepancy analysis used to refine later prompt versions.

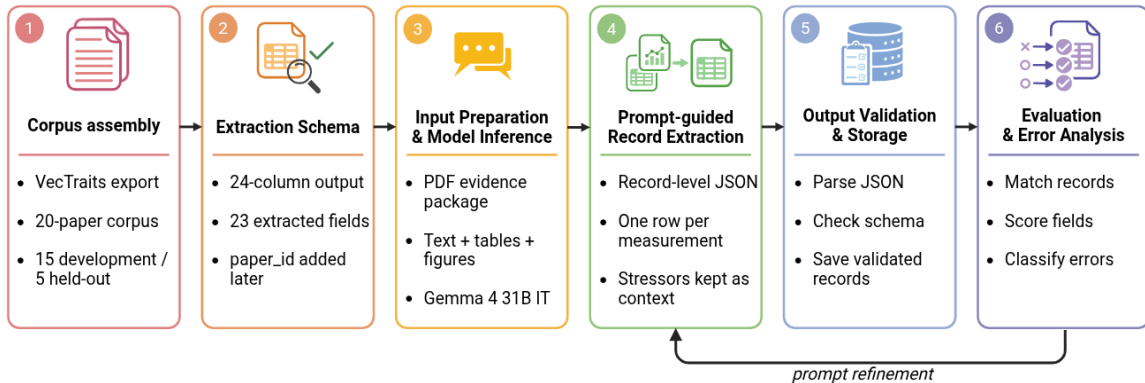


Figure 1: Overall methods workflow for prompt-engineered extraction and evaluation of VecTraits-style trait records from scientific PDFs. The workflow links corpus assembly, schema definition, input preparation, LLM-based record extraction, output validation and evaluation against a manual gold benchmark. The dashed arrow indicates the feedback loop through which evaluation errors informed later prompt refinement.

2.1 Corpus assembly

The source corpus was assembled from the VecTraits export downloaded on 27 April 2026. Records were grouped into paper-level units using DOI where available and the citation field where DOI was missing. Each paper-level unit was then assigned a stable internal identifier in the format `paper_001`, `paper_002`, and so on. These identifiers were used throughout the project to link PDFs, model outputs, reference records, evaluation outputs and log files. Papers for which the relevant curated data were primarily available in supplementary data files were removed from the sampling, because the curator would normally use the original supplementary tables rather than manually extracting values from the main PDF. Papers were also excluded if the PDF was unavailable or otherwise unsuitable for the extraction workflow.

Processing all candidate papers was not feasible within the scope of this project, because the evaluation required manual checking against the original source papers. A 20-paper corpus was therefore selected. This size was chosen to make manual gold-standard curation and disagreement review practical, while still covering a range of biological trait types, reporting formats, dataset sources and citation groups. The aim was not to process every paper in the VecTraits export, but to construct a smaller evaluation corpus that represented the main types of extraction difficulty encountered in VecTraits curation.

A light stratified sampling procedure was used to select the 20 papers. The procedure first selected seed papers to ensure coverage of primary trait families. For each

primary trait family, the highest-row candidate paper was selected. A second seed step then selected the highest-row remaining candidate for each reporting format not already represented. These seed steps ensured that the corpus included both biological diversity and variation in how trait data were presented in PDFs.

The remaining corpus slots were filled using a greedy diversity heuristic, meaning that papers were added iteratively according to the new coverage they contributed at each selection step. This follows the general greedy-selection principle of choosing the candidate with the largest immediate marginal gain at each step [Nemhauser et al., 1978, Cormen et al., 2009]. After the initial seed papers had been selected, each remaining candidate paper was scored against the corpus already selected. The highest-scoring paper was then added, the represented trait families, reporting formats, dataset sources and citation groups were updated, and the process was repeated until 20 papers had been selected. This allowed the corpus to be built transparently while prioritising diversity across the main features relevant to PDF-based trait extraction.

$$\begin{aligned} \text{Score}_i = & 4 \times \text{NewPrimaryFamily}_i + 2 \times \text{AdditionalFamilies}_i + 3 \times \text{NewFormat}_i \\ & + 1.5 \times \text{NewDataset}_i + 1.5 \times \text{NewCitation}_i + 0.75 \times \text{Size}_i. \end{aligned} \quad (1)$$

Here, $\text{NewPrimaryFamily}_i$, NewFormat_i , NewDataset_i and NewCitation_i indicated whether paper i added a primary trait family, reporting format, primary dataset source or citation group not already represented in the selected corpus. $\text{AdditionalFamilies}_i$ counted how many further trait-family labels from the same paper were also new to the corpus, while Size_i was a capped row-count term based on the number of existing `VecTraits` rows associated with the paper. The weights were chosen to prioritise biological and reporting-format diversity over simply selecting the largest papers: primary trait family received the highest weight because the benchmark needed to cover different types of vector traits, while reporting format was strongly weighted because tables, figures, mixed formats and text-heavy results pose different challenges for PDF extraction. Dataset source and citation group were included to reduce over-representation of closely related records, and row count was included only as a low-weight size term so that larger papers contributed useful evaluation material without dominating the selection.

After the 20-paper corpus was selected, it was divided into 15 development papers and five held-out papers. The held-out subset was selected using a similar greedy diversity procedure. Development papers were used for prompt iteration, extraction-rule refinement and review workflow testing. Held-out papers were reserved for final evaluation so that the final prompt could be assessed on papers that had not been used during prompt development.

2.2 Extraction schema

The target output was defined as a structured JSON schema using a reduced set of `VecTraits`-style fields. Although the full `VecTraits` export contained many columns, not all fields were suitable or necessary as direct model extraction targets. The extraction schema was therefore selected using MIREVTD as guidance, with the aim of capturing the core information needed to describe reusable vector trait records from PDFs while excluding database management fields, standardised ontology fields and highly specific optional fields [Ryan et al., 2025].

The final reduced output table contained 24 columns: one pipeline-added linking field, `paper_id`, and 23 model-extracted fields. The `paper_id` field was not extracted

from the PDF by the model. Instead, it was added by the extraction pipeline to link each output row to its source paper. All remaining fields were treated as model extraction targets.

Each row represented one paper-reported trait record. A record was defined as a single measurement bundle in which the trait name, definition, value, unit, uncertainty, organism identity, experimental condition and source in the PDF referred to the same reported measurement. The main extracting field for each record was `original_trait_name`; other fields were left empty when the information was not clearly reported for that specific record.

The 23 model-extracted fields covered study context, trait description, reported value and unit, uncertainty and sample size, primary organism identity, relevant interacting organisms, axes of variation, PDF provenance and extraction notes. Experimental and environmental conditions were represented using the shared `stressor`, `stressor_def`, `stressor_value` and `stressor_unit` fields. This general structure was used instead of separate temperature-specific fields so that temperature, humidity, diet, insecticide exposure, host type and other treatment conditions could be represented consistently. Where more than one condition applied to the same trait record, the main condition was recorded in the stressor fields and additional information was recorded in `notes`. The `notes` field was also used for ambiguity, assumptions or source-specific comments needed to interpret an extracted row.

Table 1: Reduced MIREVTD-aligned extraction schema used for model output. The schema was selected from the full VecTraits column set to capture the core information needed for reusable vector trait records, while avoiding database management fields, standardised fields and highly specific optional fields.

MIREVTD-related information	Extraction field(s)	Reason for inclusion
Study context	<code>location</code> ; <code>lab_field</code>	Records where the study was carried out and whether the measurement came from laboratory or field conditions.
Trait description	<code>original_trait_name</code> ; <code>original_trait_def</code>	Identifies the measured trait and records how the source paper defined or described it.
Trait value and unit	<code>original_trait_value</code> ; <code>original_trait_unit</code>	Records the reported value and original unit needed to interpret and reuse the trait record.
Error, variation and sample size	<code>original_error_pos</code> ; <code>original_error_neg</code> ; <code>original_error_unit</code> ; <code>interactor1number</code>	Captures reported uncertainty and the number of primary organisms used in the measurement, where available.

Continued on next page

Table 1: Reduced MIREVTD-aligned extraction schema continued.

MIREVTD-related information	Extraction field(s)	Reason for inclusion
Primary organism	<code>interactor1genus;</code> <code>interactor1species;</code> <code>interactor1stage;</code> <code>interactor1sex</code>	Identifies the organism being measured and records key biological details such as life stage and sex.
Second organism, where relevant	<code>interactor2genus;</code> <code>interactor2species;</code> <code>interactor2stage</code>	Records another organism involved in the measured trait, such as a host, prey or pathogen.
Axes of variation	<code>stressor;</code> <code>stressor_def;</code> <code>stressor_value;</code> <code>stressor_unit</code>	Records the experimental or environmental condition under which the trait was measured, such as temperature, humidity, diet, insecticide exposure, host type or another treatment.
PDF provenance and extraction notes	<code>figure_table;</code> <code>notes</code>	Records where the value was reported in the PDF and any additional information needed to interpret the row.

Full definitions and examples for all fields are provided in Appendix A.

2.3 Input preparation and model inference

The principal extraction model was Gemma 4 31B IT, implemented as `google/gemma-4-31B-it`. This model was selected because its capabilities matched the main technical requirements of the extraction task: Gemma 4 is an open, instruction-tuned multimodal model family that can process text and image inputs and generate text outputs, making it suitable for extracting structured information from both textual and visual evidence in scientific papers [Google AI for Developers, 2026]. The 31B dense variant also supports a long context window of 256K tokens, which was useful for processing paper-level inputs containing article text, tables, captions and figure evidence. Compared with smaller or more efficiency-oriented variants, the 31B dense model was therefore a better fit for this study, where extraction quality, multimodal interpretation and record-level reasoning were prioritised over low-latency inference.

The model did not accept PDF files directly, so each selected PDF was first converted into a model-readable evidence package. In this study, an evidence package refers to the collection of textual and visual information extracted from a paper and provided to the model as input. The pipeline used a text-first multimodal strategy: native PDF text was extracted where possible using PyMuPDF, so that ordinary article text did not have to be visually re-read by the model from rendered page images. If a PDF or page did not contain a usable text layer, an OCR fallback was used before text extraction, and the

resulting OCR-derived text was then handled through the same input workflow.

Tables and figures were included as part of the same evidence package. Tables were extracted as structured, markdown-formatted text where possible, so that row and column relationships were preserved together with their table labels and captions. Figures were represented using their labels, legends and cropped figure images, allowing plotted data to be interpreted in relation to the surrounding article text. Layout-aware parsing tools, such as Docling, were used where useful to identify tables, figures, captions and page regions. For visually complex tables that could not be reliably represented as text, table regions could also be cropped and included as image evidence. For each paper, the resulting article text, table content, table captions, figure legends and cropped visual evidence were combined into a single paper-level evidence package.

This design meant that the model could use article context, tables, figure legends and figure crops together when deciding which trait records to extract. The aim was not to pre-filter records by source type before model inference, but to provide the model with the relevant evidence needed to extract both table-derived and figure-derived trait records under the same schema. Tables and figures were therefore treated as equal evidence sources for extraction, rather than as separate extraction tasks.

During model inference, the evidence package was combined with the extraction prompt and the target JSON schema. Textual evidence, including the prompt, schema instructions, article text and table text, was tokenised and represented internally as text embeddings. Visual evidence, such as cropped figures or visually complex tables, was processed into image-derived embeddings or visual tokens, following the general principle that images can be represented as sequences of visual patches or features for transformer-based processing [Dosovitskiy et al., 2021, Liu et al., 2023]. These textual and visual representations were then combined within the multimodal model, allowing generation to be conditioned on both document text and visual evidence.

The model generated the output as structured text through schema-guided conditional generation. Each output token was predicted sequentially, conditioned on the prompt, the target schema, the evidence package and the tokens already generated, following the autoregressive next-token prediction framework used by transformer language models [Vaswani et al., 2017]. The schema guided the expected output structure by specifying the fields to be returned, while the document evidence provided the source information from which field values were inferred. The model therefore did not apply hand-coded extraction rules or store values in predefined variables. Instead, it generated schema-constrained JSON records using patterns learned during pre-training and instruction tuning [Wei et al., 2022, Ouyang et al., 2022].

Structured JSON records were produced by combining model generation with prompt instructions and post-generation schema validation, rather than by treating the model output as automatically valid database records. This distinction was important because the LLM output was used as a candidate extraction result that required validation before evaluation or curation use.

Model inference was configuration-driven. The model configuration, prompt version, input directory, output directory and run name were specified at run time and recorded with the run outputs. This made each extraction run traceable to the model backend and instruction set used. Multiple PDFs could be processed as independent paper-level jobs within the same run, while keeping each paper as a separate unit of extraction. The pipeline also supported retrying failed model calls and resuming runs by skipping papers that had already produced successful record files.

2.4 Prompt-guided record extraction

Prompt versions were retained rather than overwritten and were selected at run time through the model configuration. Prompt design was revised iteratively after reviewing extraction errors. Earlier prompts produced overly broad or fragmented outputs, so later versions added stricter instructions for record-level extraction.

The model was instructed to extract complete paper-reported trait records rather than isolated trait-like phrases. Each record had to keep the trait name, definition, value, unit, uncertainty, organism identity, experimental condition and figure or table source together when these fields referred to the same measurement. When the same trait was reported under different temperatures, species, stages, treatments or locations, the prompt instructed the model to return separate records rather than merge them into one row.

The prompt also instructed the model to use all available evidence sources together. For figure-derived data, the model was asked to use the figure legend, axis labels, tick marks, units and article context to decide whether a plot contained extractable trait values. Values were to be digitised only when the plotted bars, points or lines were clearly readable. If a plotted value could not be read confidently, the model was instructed to return a missing value and explain the uncertainty in **notes**.

Additional constraints were used to reduce over-extraction and field mixing. The model was instructed to preserve author-reported wording where possible, return missing optional fields as null, avoid inventing values or units, and exclude general background statements, framework terms, malformed fragments and expressions belonging to other schema fields. Temperature and other experimental or environmental conditions were recorded using the shared stressor fields rather than as trait names. If more than one condition applied to the same record, the main condition was placed in **stressor**, **stressor_def**, **stressor_value** and **stressor_unit**, while additional condition information was recorded in **notes**. The **figure_table** field was used to record the reported source of each extracted value, such as Table 1, Figure 2, or Result text.

2.5 Output validation and storage

All prompts required JSON-only output following the target schema so that responses could be parsed and validated automatically. After inference, model responses were first cleaned for parsing by removing Markdown code fences where present. The remaining text was then parsed as JSON and checked for a valid top-level **records** list. Each record was checked against the expected reduced extraction schema, and outputs that could not be parsed or that failed schema validation were logged for inspection.

After validation, records were normalised before being written to the run output table. This included trimming whitespace, filling missing optional fields with empty values, and removing records without **original_trait_name**. The **paper_id** field was then added by the pipeline using the corpus manifest or source filename, rather than being extracted by the model.

Validated records were saved as file-based run outputs so that each extraction experiment had a reproducible archive. The validated output table was then used for downstream comparison with the manual gold benchmark reference.

2.6 Evaluation and error analysis

Evaluation combined automated record matching, field-value scoring and qualitative error analysis. The aim was to assess whether the model recovered the correct paper-reported trait records and whether the values within those records were accurately extracted. Manual discrepancy analysis was used to interpret recurring errors and guide prompt refinement.

The manual gold benchmark was used as the reference for the quantitative evaluation. This benchmark was constructed by re-curating the selected source papers under the reduced extraction schema, and was treated as an improved version of the corresponding VecTraits records for the selected papers. Compared with the original VecTraits export, it was designed to be more complete, more accurate and cleaner for evaluation purposes. It included additional paper-reported trait records where relevant, removed unsupported or low-value rows, preserved original trait names and definitions more closely where possible, and applied more consistent rules for extracting field values from the source papers. Because reference quality directly affects evaluation quality, the unmodified VecTraits export was not used as the main quantitative reference.

For each paper, model records were first matched to gold benchmark records within the same paper. Row-set metrics were then used to quantify whether the model extracted the correct set of paper-reported records, while field-level metrics were used to assess the correctness of individual extracted values within matched records. Qualitative error analysis was then used to identify recurring causes of extraction failure, comparison difficulty and prompt weakness.

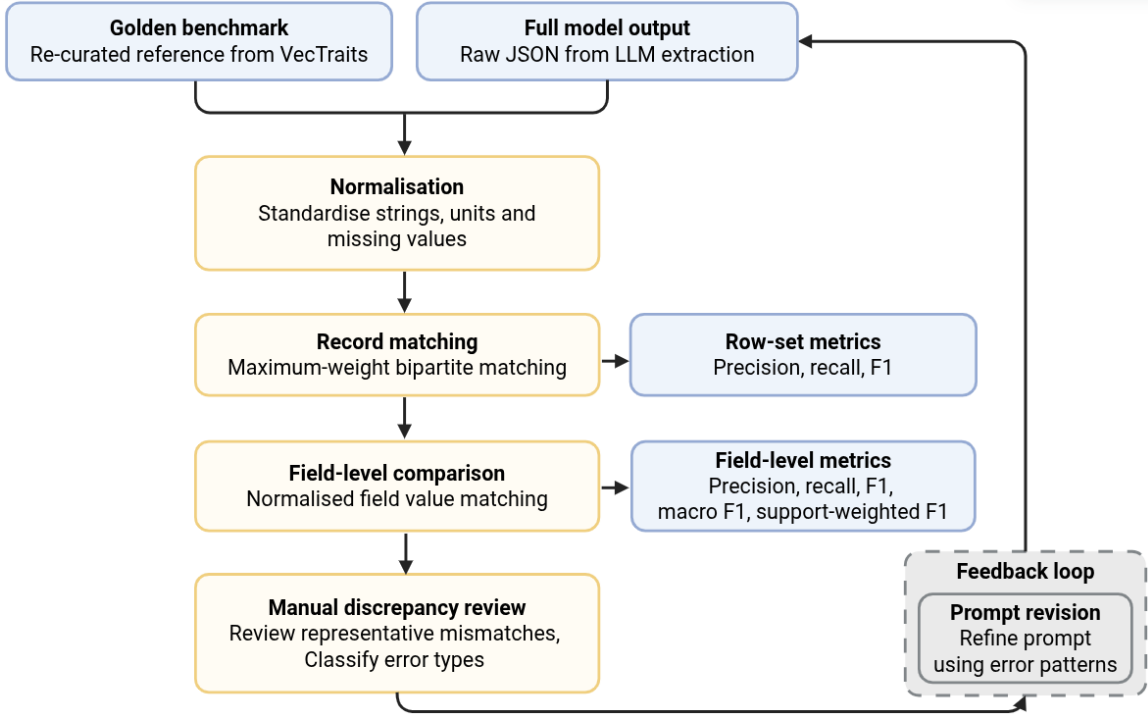


Figure 2: Evaluation workflow for assessing LLM-extracted trait records against the manual gold benchmark. The full model output and the gold benchmark were first normalised to standardise strings, units and missing values. Model and reference records were then aligned within each paper using maximum-weight bipartite record matching. Matched records were passed to field-level comparison, where individual extracted values were evaluated using normalised field-value matching. Row-set metrics summarised whether the model extracted the correct set of records, while field-level metrics summarised the accuracy of values within matched records. Representative mismatches were then reviewed manually to classify recurring error types and guide prompt revision in the next extraction run. Blue boxes indicate inputs and quantitative outputs, yellow boxes indicate evaluation workflow steps, and the grey dashed box indicates the prompt-improvement feedback loop.

2.6.1 Record matching

Before field-value scoring, model records were aligned with reference records within the same paper using one-to-one bipartite record matching. This step was needed because model outputs and reference tables did not necessarily contain records in the same order, and the number of model-extracted records could differ from the number of reference records.

For each paper, all possible model-reference record pairs were assigned a similarity score based on normalised agreement across identifying fields. These fields included `original_trait_name`, `original_trait_value`, `original_trait_unit`, `figure_table`, `interactor1species`, `interactor1stage`, `stressor_value` and `stressor_unit`. The trait name contributed most strongly to the pair score because it was the most discriminating field for identifying what was measured, whereas contextual fields such as species, stage or treatment could be shared by many records within the same paper.

The resulting similarity matrix was then used to solve a maximum-weight one-to-one assignment problem [Crouse, 2016]. This selected the set of model-reference pairs

that maximised the total matching score while ensuring that each model record and each reference record was used at most once. Candidate pairs below the minimum matching threshold were left unmatched. Matched pairs were passed to field-value scoring. Model records without a reference match were counted as extra model records, and reference records without a model match were counted as missing reference records.

2.6.2 Field-value comparison

After record matching, values within matched records were compared field by field using normalised field-value matching. The `paper_id` field was not evaluated as a model-extracted field because it was added by the pipeline for linking each record to its source paper. The `notes` field was retained for interpretation but excluded from the main quantitative F1 calculations because acceptable notes could vary in wording and functioned mainly as supporting information rather than as a biological trait value.

Before comparison, model and reference values were normalised in memory. This normalisation was applied only during evaluation and did not modify the original model output files. Text values were converted to lower case, leading and trailing whitespace was removed, repeated spaces were collapsed, and common missing-value strings such as empty strings, `na`, `n/a`, `null`, `none` and `-` were mapped to empty values. Unit fields were standardised using an alias table, so that equivalent forms such as `°C`, `degrees Celsius` and `Celsius` were treated as the same unit. Numeric fields were parsed to extract comparable numeric values before matching, so that minor formatting differences did not create false mismatches. Life stage, sex and study-setting fields were also standardised using simple aliases where expected.

After normalisation, a field was counted as matching when the model and reference values represented the same reported information. For most short categorical fields, this required equality after normalisation. For longer descriptive fields, such as `location`, `original_trait_def` and `stressor_def`, relaxed matching was allowed when one value preserved a longer source-paper phrase and the other gave a shorter but equivalent description. Borderline cases were retained for qualitative discrepancy analysis.

For each scored field j , a true positive TP_j was recorded when both model and reference values were non-empty and matched after normalisation. A false negative FN_j was recorded when the reference contained a value that the model missed or mismatched. A false positive FP_j was recorded when the model produced a value that was absent from the reference or mismatched the reference value. Cases where both model and reference values were empty were recorded as true negatives but excluded from precision and recall.

2.6.3 Summary metrics

The main automated outputs were row-set metrics and field-level metrics. Row-set metrics evaluated whether the model extracted the correct set of records, while field-level metrics evaluated whether the values inside matched records were correct.

Row-set precision, recall and F1 were calculated from the number of matched, extra and missing records:

$$\text{Row-set Precision} = \frac{n_{\text{matched}}}{n_{\text{matched}} + n_{\text{extra}}} \quad (2)$$

$$\text{Row-set Recall} = \frac{n_{\text{matched}}}{n_{\text{matched}} + n_{\text{missing}}} \quad (3)$$

$$\text{Row-set F1} = \frac{2 \times \text{Row-set Precision} \times \text{Row-set Recall}}{\text{Row-set Precision} + \text{Row-set Recall}} \quad (4)$$

Here, n_{matched} is the number of model records matched to reference records, n_{extra} is the number of unmatched model records, and n_{missing} is the number of unmatched reference records. Row-set precision therefore penalised over-extraction, while row-set recall penalised under-extraction.

Field-level precision, recall and F1 were calculated separately for each scored field after record matching:

$$\text{Precision}_j = \frac{TP_j}{TP_j + FP_j} \quad (5)$$

$$\text{Recall}_j = \frac{TP_j}{TP_j + FN_j} \quad (6)$$

$$F1_j = \frac{2 \times \text{Precision}_j \times \text{Recall}_j}{\text{Precision}_j + \text{Recall}_j} \quad (7)$$

Here, TP_j , FP_j and FN_j are the field-level counts for scored field j , as defined during field-value comparison. Fields with no true positive, false positive or false negative signal in a given run were excluded from the summary field-level averages.

Macro field F1 was calculated as the unweighted mean of per-field F1 scores:

$$\text{Macro Field F1} = \frac{1}{n} \sum_{j=1}^n F1_j \quad (8)$$

In this equation, n is the number of quantitatively scored fields included in the calculation. Macro field F1 treated each scored field equally, regardless of how often it appeared in the reference data.

Support-weighted field F1 was calculated as a support-weighted average of per-field F1 scores:

$$\text{Support-weighted Field F1} = \frac{\sum_{j=1}^n s_j F1_j}{\sum_{j=1}^n s_j} \quad (9)$$

Here, s_j is the support for field j , defined as the number of non-empty reference values for that field. Support-weighted field F1 therefore summarised how accurately the model filled fields within matched records while giving greater influence to fields that were more commonly reported in the reference data.

2.6.4 Qualitative discrepancy analysis

Automated scores were supplemented with qualitative discrepancy analysis. This step was used to understand why model and reference records disagreed, rather than to replace or adjust the automated metrics. Discrepancies included unmatched model records, unmatched reference records and field-level mismatches within matched pairs.

Reviewed discrepancies were grouped into broad error categories, including missing records, unsupported extra records, acceptable paper-reported records absent from the reference, field-value mismatches, unit or formatting differences, row-alignment problems, figure-digitisation uncertainty and possible reference omissions. These categories helped

distinguish genuine extraction errors from cases where the model output was supported by the source paper but not represented in the comparison reference.

Findings from discrepancy analysis were used to guide prompt refinement and to identify recurring limitations in the extraction workflow. For example, repeated missing records indicated under-extraction, unsupported extra records indicated over-extraction, and figure-digitisation uncertainty highlighted cases where plotted values were difficult to recover reliably. Manual discrepancy analysis was not used to flip match labels or recalculate adjusted precision, recall or F1. The final reported quantitative results therefore remained the automated row-set and field-level scores.

3 Results

4 Discussion and Conclusion

Although the original VecTraits export was not used as the main quantitative reference, comparison with the prompt-engineered extraction output may still be useful for future VecTraits curation. The manually checked benchmark suggested that some existing VecTraits records could be made more complete, more accurate and closer to the original source-paper wording. In this sense, prompt-engineered extraction could support future curation by helping identify missing paper-reported records, unsupported rows, inconsistent original trait names and field values that require checking. Rather than treating the existing VecTraits data as a gold standard for model evaluation, the more useful future direction may be to use evaluated extraction outputs as a tool for improving the originality, consistency and traceability of VecTraits records.

References

- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. Language models are few-shot learners. *arXiv*, 2020. doi: 10.48550/arXiv.2005.14165.
- Lauren Cator, Samraat Pawar, William Pearse, S. White, S. Modrak, F. Windram, S. Kelly, R. Jones, H. Vineer, M. England, and C. Sanders. Vbd hub: Open-source platform for vector-borne disease data integration, 2026.
- Lauren J. Cator, Leah R. Johnson, Erin A. Mordecai, Fadoua El Moustaid, T. R. C. Smallwood, Shannon L. LaDeau, Michael A. Johansson, Peter J. Hudson, Michael Boots, Matthew B. Thomas, Alison G. Power, and Samraat Pawar. The role of vector trait variation in vector-borne disease dynamics. *Frontiers in Ecology and Evolution*, 8, 2020. doi: 10.3389/fevo.2020.00189.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- David F. Crouse. On implementing 2d rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52(4):1679–1696, 2016. doi: 10.1109/TAES.2016.140952.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain

- Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Google AI for Developers. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4, 2026. Accessed 2026-06-23.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.
- Joshua S. Madin, Shawn Bowers, Mark P. Schildhauer, and Matthew B. Jones. Advancing ecological research with ontologies. *Trends in Ecology & Evolution*, 23(3):159–168, 2008. doi: 10.1016/j.tree.2007.11.007.
- Zara Nasar, Syed Waqar Jaffry, and Muhammad Kamran Malik. Information extraction from scientific articles: a survey. *Scientometrics*, 117(3):1931–1990, 2018. doi: 10.1007/s11192-018-2921-5.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- Sadie J. Ryan, Paul J. Huxley, Catherine A. Lippi, Samraat Pawar, Lauren Cator, Samuel S. C. Rund, and Leah R. Johnson. Mirevtd, a minimum information standard for reporting vector trait data, 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3(1):160018, 2016. doi: 10.1038/sdata.2016.18.

Table 2: Selected 20-paper corpus used for prompt development and evaluation. The table summarises the development or held-out split, existing VecTraits row count, inferred primary trait family, inferred reporting format, dataset source and selection rationale. Trait family and reporting format were used as sampling descriptors rather than definitive biological classifications.

Paper ID	Split	VecTraits rows	Primary trait family	Reporting format	Dataset source	Selection rationale
paper_028	development	112	behaviour_movement	table-heavy	537	Selected as the highest-row representative of the behaviour/movement primary trait family.
paper_036	development	220	development	table-heavy	653	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.
paper_058	development	326	survival_mortality	figure-heavy	505	Selected as the highest-row representative of the survival/mortality primary trait family.
paper_074	development	2840	development	mixed	1228	Selected as the highest-row remaining representative of mixed-format reporting.
paper_082	development	272	development	mixed	1513	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.

Continued on next page

Table 2: Selected 20-paper corpus used for prompt development and evaluation. Continued.

Paper ID	Split	VecTrait rows	Primary trait family	Reporting format	Dataset source	Selection rationale
paper_096	development	280	development	table-heavy	318	Selected as the highest-row remaining representative of table-heavy reporting.
paper_142	development	268	longevity	mixed	564	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.
paper_150	development	238	development	table-heavy	354	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.
paper_180	development	234	development	table-heavy	113	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.
paper_234	development	411	physiology_environment	figure-heavy	539	Selected as the highest-row representative of the physiology/environmental response primary trait family.

Continued on next page

Table 2: Selected 20-paper corpus used for prompt development and evaluation. Continued.

Paper ID	Split	VecTrai rows	Primary trait family	Reporting format	Dataset source	Selection rationale
paper_239	development	40	body_size_morphology	table-heavy	391	Selected as the highest-row representative of the body size/morphology primary trait family.
paper_252	development	208	development	table-heavy	481	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.
paper_262	development	343	longevity	figure-heavy	634	Selected as the highest-row remaining representative of figure-heavy reporting.
paper_298	development	46	feeding_behavior	table-heavy	661	Selected as the highest-row representative of the feeding behaviour primary trait family.
paper_321	development	216	development	table-heavy	1529	Selected by the diversity score to add a new dataset source and citation group while retaining a high row count.

Continued on next page

Table 2: Selected 20-paper corpus used for prompt development and evaluation. Continued.

Paper ID	Split	VecTrai rows	Primary trait family	Reporting format	Dataset source	Selection rationale
paper_089	held-out	7908	development	mixed	223	Selected as the highest-row representative of the development primary trait family and assigned to the held-out split to retain evaluation diversity.
paper_152	held-out	356	other_uncategorised	table-heavy	436	Selected as the highest-row representative of the other/uncategorised primary trait family and assigned to the held-out split to retain trait-family diversity.
paper_217	held-out	671	longevity	table-heavy	124	Selected as the highest-row representative of the longevity primary trait family and assigned to the held-out split to retain evaluation diversity.
paper_231	held-out	7	development	text-heavy	856	Selected as the highest-row remaining representative of text-heavy reporting and assigned to the held-out split to retain reporting-format diversity.

Continued on next page

Table 2: Selected 20-paper corpus used for prompt development and evaluation. Continued.

Paper ID	Split	VecTraitor rows	Primary trait family	Reporting format	Dataset source	Selection rationale
paper_329	held-out	1024	reproduction_fecundity	figure heavy	965	Selected as the highest-row representative of the reproduction/fecundity primary trait family and assigned to the held-out split to retain evaluation diversity.

A VecTraits extraction schema field definitions

Table 3: VecTraits extraction schema field definitions.

Field	Plain English name	Definition	Example
Model extraction target fields			
location	Study location	Geographic location reported for the study, population or experiment.	Kampala, Uganda
lab_field	Study setting	Whether the study was conducted under laboratory or field conditions.	laboratory
original_trait_name	Trait name	The exact name of the trait as reported in the source paper, without standardisation or renaming.	development time
original_trait_def	Trait definition	The definition or description of the trait as stated or implied in the source paper.	time from egg hatching to adult emergence
original_trait_value	Trait value	The numeric or categorical value of the trait as reported in the source paper.	13.8
original_trait_unit	Trait unit	The original unit of measurement reported in the source paper.	days
original_error_pos	Positive error	The positive component of the reported uncertainty, such as the upper bound of a range or the positive value in plus-minus notation.	0.5
original_error_neg	Negative error	The negative component of the reported uncertainty, such as the lower bound of a range or the negative value in plus-minus notation.	0.5
original_error_unit	Error unit	The unit of the reported error values.	days
interactor1genus	Primary genus	Genus of the primary organism, usually the vector species being measured.	Aedes

Continued on next page

Table 3: VecTraits extraction schema field definitions. continued

Field	Plain English name	Definition	Example
interactor1species	Primary species	Species of the primary organism, usually the vector species being measured.	aegypti
interactor1stage	Primary life stage	Life stage of the primary organism at the time of measurement.	larva
interactor1sex	Primary sex	Sex of the primary organism, where reported.	female
interactor1number	Number of primary organisms	The number of primary organisms used in the measurement, where this was reported.	30
interactor2genus	Secondary genus	Genus of the second interacting organism, such as a host, prey or pathogen, where relevant.	Plasmodium
interactor2species	Secondary species	Species of the second interacting organism, where relevant.	falciparum
interactor2stage	Secondary life stage	Life stage of the second interacting organism, where relevant.	sporozoite
stressor	Stressor or treatment name	The experimental or environmental condition under which the trait was measured, such as temperature, humidity, diet, insecticide, host type or another treatment.	temperature
stressor_def	Stressor definition	Description of the stressor or treatment as reported in the paper.	constant rearing temperature
stressor_value	Stressor value	Numeric or categorical value of the stressor or treatment.	28
stressor_unit	Stressor unit	Unit of the stressor value, where applicable.	°C
figure_table	Source figure or table	The figure, table or text location in the source paper from which the value was extracted.	Table 2
notes	Extraction notes	Extra information needed to interpret the extracted row, such as ambiguity, assumptions or source-specific comments.	value read from figure caption

Continued on next page

Table 3: VecTraits extraction schema field definitions. continued

Field	Plain English name	Definition	Example
Linking field, not a model extraction target			
<code>paper_id</code>	Paper identifier	Stable internal identifier assigned to each source paper in this study. This field was added during corpus assembly and was not extracted by the model.	<code>paper_010</code>

Table 4: Field-specific normalisation rules used before automated field-value matching.

Field type	Normalisation rule
Short text fields	Lower case, trim whitespace, collapse repeated spaces, standardise missing values.
Unit fields	Apply unit alias table before comparison.
Numeric fields	Parse comparable numeric values before comparison.
Long descriptive fields	Apply relaxed matching where wording differed but meaning was equivalent.